

Elena Revicheva

AI Automation Engineer · case studies for Ripio

Panama City (UTC-5) · aideazz.xyz/portfolio · Incident record: aideazz.xyz/ai-ops-wiki.html · Python · TypeScript · LLM APIs · GitHub Actions

I design, build and run a fleet of LLM agents and automations alone, on one Oracle Cloud server, for my own AI lab. Each one replaced manual work I was doing by hand. I build **AI-native**: I frame the problem, delegate the implementation to coding agents (Claude Code, Cursor), and I own the correctness of what comes back. That last part takes most of my hours, and your posting asks for it directly. Before this I was Deputy CEO and Chief Legal Officer in e-government for seven years, so I start from the business process and the compliance risk, not from the tool.

YOUR REQUIREMENTS → WHERE I HAVE DONE IT

RIPIO ASKS FOR	EVIDENCE, IN PRODUCTION
Find and automate manual processes	Job search, CRM hygiene, lead response, daily publishing and a morning status briefing all used to be done by hand and now run unattended. I pick targets by measuring the hours spent, and I remove automation when it does not pay off (case 2).
Agents and bots on LLMs, integrated with chat, internal APIs and third parties	A Telegram control plane with 110+ commands, WhatsApp and Telegram assistants, and integrations with the HubSpot CRM, Resend email, GitHub (the bot opens real pull requests) and Bright Data. My chat surface is Telegram and WhatsApp; the same pattern applies to Slack.
Business need before the solution	Each system starts as a written problem statement. Example: the ad-intelligence tool is "detect, not predict": it reports only what can be checked, because a forecast cannot be verified.
End to end, CI/CD, production monitoring	A GitHub Actions deploy workflow covers 12 services plus a fleet-verify job. PM2 process supervision. Scheduled read-back audits that alert on Telegram. A synthetic lead goes through the real pipeline every morning and alerts only if it fails.
AI in the dev loop: context, harness, agent self-correction	A 534-test eval suite on the job engine, with offline layers that run before every rule change. A shared-memory protocol lets two coding agents that cannot see each other work one repo safely. A generation gate retries and salvages instead of publishing unverified figures (case 3).
Work with Infra and Security	Attachment gate with 24 tests (path traversal, magic bytes, a file claiming the wrong type). CRM files stored private. Leak scan before anything is published. Credentials never pass through the agents. No irreversible action without a human click.
Backend languages	Python (LangGraph, FastAPI) and TypeScript in production. Repos: 1,640 commits (ops agent), 578 (job engine), 3,513 (web and content engine).

CASE STUDIES

1 · Inbound lead concierge: form → CRM → drafted reply → one-tap approval

TypeScript · HubSpot API · Telegram · Resend · webhook API

A form submission becomes a CRM record and an LLM-drafted reply, which arrive on my phone as an approval card. One tap sends it, and delivery and open events are written back to the deal. Two systems can write the reply, and redundancy alone means the faster one always wins. So a **cached health verdict decides precedence**, with a 5-minute grace period and a 20-minute safety net. The approval card is the product: a human stays on the last click of anything that reaches a customer.

2 · Job-discovery engine: LangGraph, LLM judge, human interrupt, outcome feedback

Python · LangGraph · SQLite checkpointing · pytest

- State machine: gate → score → route → notify, checkpointed so a crash resumes instead of restarting.
- An LLM judge scores fit, and a **trust clamp** limits how far keyword rules can overrule it. That clamp fixed an incident where keyword bonuses were silently outvoting a correct "no".
- A feedback loop reads what I actually did in the CRM, including my typed reasons, and writes it back into the judge's prompt. It **fails closed** when the LLM is down: no score is better than a made-up one.
- It originally auto-applied. I measured the result, **volume and zero outcomes**, and removed that feature.

3 · Correctness of agent output: the part I treat as the real job

Harness · context management · self-correction

- **Grounded generation:** the daily article generator collects production evidence first (commits, uptimes, CRM counts). A gate rejects any figure it cannot trace, then retries up to 4 times with a different angle, and salvages sourced paragraphs instead of discarding the draft.
- **Context across agents:** two coding agents work the same repo without seeing each other's chats. A shared file acts as their working memory: a claim board before touching files, a handoff block (done / next / verified by / risk), and a start-of-session check that the last change produced output, not just that it ran.
- **Read-back verification.** This week an audit that reads the CRM back showed a pipeline reporting success while only 11 of 23 job records had a real cover letter. A refused API call was being logged as "no data". I added retry-on-429, made failed lookups raise, added a self-healing job every 2 hours and a daily audit with alerts. Result: 19 of 23 complete, and the remaining 4 proven to be closed postings.

4 · Five-provider LLM chain, ordered per use case

Claude · OpenAI · Gemini · Groq · Grok

One API key is a single point of failure for the whole operation. On 19 Aug 2026 my primary provider's balance hit zero, and the chain fell through to the next provider in 2.4 seconds with no lost request. The lesson that took longer: **the order must differ by use case**. Reasoning models can return empty strings under low token budgets, so a classification chain that leads with one fails silently. Client-facing copy leads with the strongest model; classification leads with the cheapest one that clears the bar.

5 · Crypto-adjacent work

DragonTrade: a trading-education agent that runs on paper trading only. I built it to learn how to run an agent that must never take an irreversible financial action without a human. **ATUONA:** an NFT art gallery I designed and built, with wallet-connect drops and an automated AI film pipeline behind it (8 films live).

PRINCIPLES I WOULD BRING TO RIPIO'S INTERNAL TOOLING

Measure at the edge, not the source	A commit is not a deploy, and an accepted email is not a delivered one. Verify where the request actually lands.
Unmeasured is not zero	A tracker exits non-zero when it cannot measure, so "nothing counted" never looks like "nothing happened".
Fail open or closed, per system	Alerting fails open: a duplicate beats silence. Publishing and money fail closed.
Every record names its author	Many agents write to one CRM, and a source prefix turns provenance into a filter.
Write the failure down	A public incident record: 22 incidents and 19 named failure modes, each with the measured number that proves it.

Figures re-read from the running systems and repositories on 23 September 2026. Code on GitHub. I am happy to walk through any of it live, including the parts that broke. · Elena Revicheva · [aideazz.xyz/portfolio](#)